

Diseño Algorítmico y Plan de Validación

Cobertura de Vértices para la ubicación de sistemas de detección de intrusos

Grupo 1

Universidad Nacional de San Antonio Abad del Cusco
Escuela Profesional de Ingeniería Informática y de Sistemas

Asignatura: Algoritmos Avanzados
Docente: Héctor Eduardo Ugarte Rojas

Semestre académico 2026-2
Avance 2: Diseño algorítmico y plan de validación

Resumen

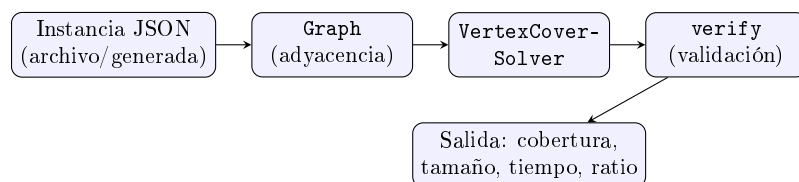
Se presenta el diseño preliminar del sistema: módulos, estructuras de datos, flujo de información, formato de archivos, pseudocódigo y análisis de complejidad de los algoritmos implementados. Se resuelve paso a paso una instancia pequeña usando el sistema real (no solo a mano), se describe el plan de pruebas (básicas, límite y adversarias) ya implementado, el criterio de verificación de salidas, las herramientas candidatas evaluadas con sus criterios de selección, y las métricas previstas para la comparación experimental. Todo el código referido en este documento corresponde al repositorio Git del proyecto y es ejecutable.

1 Arquitectura y módulos del sistema

El sistema se organiza en módulos con responsabilidades separadas, siguiendo el principio de que la lógica algorítmica no debe depender de cómo se presenta o se invoca:

Módulo	Archivo	Responsabilidad
Grafo	<code>src/graph.py</code>	Estructura de datos, carga/guardado de instancias, generadores sintéticos
Solver	<code>src/solver.py</code>	Los tres algoritmos (greedy, matching, exacto)
Verificación	<code>src/verify.py</code>	Validación de coberturas y cálculo de métricas
Pruebas	<code>tests/</code>	Casos básicos, límite y adversarios (pytest)
Experimentos	<code>experiments/compare.py</code>	Comparación empírica y exportación a CSV
Prototipo/demo	<code>demo.py</code>	Interfaz de línea de comandos para la demostración

1.1 Flujo de información



1.2 Estructura de datos elegida

Se usa una **lista de adyacencia** (`dict[vértice, set[vértice]]`) en lugar de una matriz de adyacencia. Justificación:

- Las redes reales son dispersas ($|E| \ll |V|^2$), por lo que una matriz desperdiciaría memoria: $O(V^2)$ vs. $O(V + E)$ de la lista de adyacencia.
- Obtener el grado de un vértice es $O(1)$ (tamaño del conjunto de vecinos), operación central en el algoritmo greedy.
- Insertar/eliminar una arista es $O(1)$ amortizado gracias al uso de `set`.

1.3 Formato de archivo de instancia

Las instancias se guardan en JSON, elegido por ser legible por humanos, nativo en Python y fácil de versionar en Git (diffs claros por línea):

```
{
  "name": "example_small",
  "description": "...",
  "vertices": ["A", "B", "C", "D", "E", "F"],
  "edges": [["A","B"], ["A","C"], ["A","D"], ["B","C"], ["D","E"], ["E","F"]]
}
```

2 Pseudocódigo y análisis de complejidad

2.1 Greedy por grado

Algorithm 1 greedy_degree_vertex_cover

```

1:  $E' \leftarrow E$  ▷ aristas pendientes
2:  $C \leftarrow \emptyset$ 
3: while  $E' \neq \emptyset$  do
4:   calcular  $\deg_{E'}(v)$  para cada  $v$  incidente en  $E'$ 
5:    $v^* \leftarrow \arg \max_v \deg_{E'}(v)$ 
6:    $C \leftarrow C \cup \{v^*\}$ 
7:    $E' \leftarrow E' \setminus \{(u, w) \in E' : u = v^* \vee w = v^*\}$ 
8: end while
9: return  $C$ 
```

Complejidad: en la implementación actual (`src/solver.py`), cada iteración recorre las aristas restantes para recalcular grados: $O(E)$ por iteración, hasta $O(V)$ iteraciones en el peor caso $\Rightarrow O(V \cdot E)$. Puede optimizarse a $O((V + E) \log V)$ manteniendo una cola de prioridad con grados actualizados perezosamente; queda identificado como mejora para la Entrega 2.

Garantía de aproximación: ninguna cota fija demostrable en el peor caso general (existen instancias donde el greedy se aleja más del óptimo que el matching maximal); en la práctica, sobre las instancias probadas, el ratio empírico se mantuvo entre 1.0x y 1.2x (ver Sección 7).

2.2 Matching maximal (2-aproximación)

Algorithm 2 maximal_matching_vertex_cover

```

1:  $E' \leftarrow E$ 
2:  $C \leftarrow \emptyset$ 
3: while  $E' \neq \emptyset$  do
4:   tomar  $(u, v) \in E'$  ▷ orden canónico, ver Nota de reproducibilidad
5:    $C \leftarrow C \cup \{u, v\}$ 
6:    $E' \leftarrow E' \setminus \{(a, b) \in E' : a \in \{u, v\} \vee b \in \{u, v\}\}$ 
7: end while
8: return  $C$ 

```

Complejidad: $O(V + E)$: cada arista se visita y se elimina a lo sumo una vez del conjunto de pendientes.

Garantía de aproximación (demostración breve): las aristas elegidas $(u_1, v_1), (u_2, v_2), \dots$ no comparten extremos (forman un *matching*), porque cada elección elimina todas las aristas incidentes a los vértices recién agregados. Por lo tanto, toda cobertura válida —incluida la óptima— debe contener al menos un extremo de cada una de esas aristas (ya que son disjuntas, ningún vértice puede "cubrir" dos de ellas a la vez). Si el matching tiene k aristas, entonces $|C^*| \geq k$. El algoritmo agrega $2k$ vértices, por lo tanto $|C_{\text{match}}| = 2k \leq 2 \cdot |C^*|$.

2.3 Exacto (fuerza bruta de referencia)

Algorithm 3 exact_vertex_cover_bruteforce

```

1: for  $k = 1$  hasta  $|V|$  do
2:   for cada subconjunto  $S \subseteq V$  con  $|S| = k$  do
3:     if  $S$  cubre todas las aristas de  $E$  then
4:       return  $S$ 
5:     end if
6:   end for
7: end for

```

Complejidad: $O\left(\binom{V}{k^*} \cdot E\right)$ en el peor caso, acotado superiormente por $O(2^V \cdot E)$. Por esta razón `exact_vertex_cover_bruteforce` rechaza instancias de más de 20 vértices (parámetro configurable) y lanza `ValueError` en vez de ejecutarse indefinidamente. Los datos de la Sección 7 confirman el crecimiento exponencial esperado.

Nota de reproducibilidad. Durante la implementación se detectó que iterar directamente sobre un `set()` de aristas en Python produce un orden distinto en cada ejecución del proceso (por la aleatorización del hash de los *strings*, `PYTHONHASHSEED`), lo que rompía la reproducibilidad exigida por el proyecto: dos corridas de un mismo algoritmo sobre la misma instancia podían devolver coberturas válidas pero de tamaño distinto. Se corrigió introduciendo `VertexCoverSolver._sorted_edges()`, que fija un orden canónico y determinístico para los desempates del greedy y la elección "arbitraria" del matching. Ver el commit `fix: hacer deterministas greedy y matching maximal` en el historial del repositorio.

3 Instancia resuelta paso a paso con el sistema real

Se ejecuta `demo.py` sobre la instancia `data/instances/example_small.json` (la misma mini-red de 6 routers del Avance 1):

```
$ python demo.py --instance data/instances/example_small.json --algorithm all
```

Salida real del sistema:

```
Cargado: instancia 'data/instances/example_small.json'
|V| = 6 |E| = 6
```

```
[Greedy por grado]
```

```
Cobertura: ['A', 'B', 'E']
```

```
Tamano: 3
```

```
Valida: SI
```

```
Tiempo: 0.034 ms
```

```
[Matching maximal (2-aprox)]
```

```
Cobertura: ['A', 'B', 'D', 'E']
```

```
Tamano: 4
```

```
Valida: SI
```

```
Tiempo: 0.018 ms
```

```
[Exacto (fuerza bruta)]
```

```
Cobertura: ['A', 'B', 'E']
```

```
Tamano: 3
```

```
Valida: SI
```

```
Tiempo: 0.031 ms
```

```
Ratios de aproximacion frente al optimo:
```

```
Greedy por grado: 1.000x
```

```
Matching maximal (2-aprox): 1.333x
```

El resultado coincide exactamente con la traza manual del Avance 1 (greedy = $\{A, B, E\}$, tamaño 3; matching = $\{A, B, D, E\}$, tamaño 4), lo que valida que la implementación respeta el pseudocódigo acordado.

4 Plan de pruebas

El plan de pruebas ya está implementado en `tests/` (27 casos, `pytest`) y se organiza en las tres categorías pedidas:

Categoría	Casos	Archivo
Básicos	Construcción del grafo; instancia del Avance 1 (resultado exacto esperado); triángulo K_3 (óptimo conocido = 2).	<code>test_graph.py</code> , <code>TestCasos-</code> <code>Basicos</code>
Límite	Grafo sin aristas; una sola arista (evidencia directa de la cota $2x$); vértices aislados; grafo completo K_4 (óptimo = $n - 1$); componentes desconectadas; fuerza bruta rechazando instancias > 20 vértices.	<code>TestCasosLimite</code>
Adversarios	Grafo estrella (greedy óptimo, matching subóptimo pero válido); verificación parametrizada en 10 grafos aleatorios de que el matching maximal nunca supera $2x$ el óptimo; verificación de que ninguna arista queda sin cubrir.	<code>TestCasos-</code> <code>Adversarios</code>

4.1 Criterio de verificación de salidas

Ninguna prueba confía únicamente en el *tamaño* de la cobertura devuelta. Toda salida se valida con `is_vertex_cover(graph, cover)` (`src/verify.py`), que recorre todas las aristas del grafo y confirma que cada una tenga al menos un extremo en la cobertura. Esto es intencional: un

algoritmo con un error de implementación podría devolver un conjunto del tamaño "esperado" sin que efectivamente cubra el grafo, y una prueba que solo comparara tamaños no lo detectaría.

```
$ pytest tests/ -v
...
===== 27 passed in 0.04s =====
```

5 Herramientas candidatas y criterios de selección

Necesidad	Candidatas	Criterio de selección
Pruebas unitarias	<code>pytest</code> vs. <code>unittest</code> (estándar)	<code>pytest</code> elegido: sintaxis de <code>assert</code> plano (más legible), soporte nativo de parametrización (<code>@pytest.mark.parametrize</code> , usado en las pruebas adversarias), amplia adopción y documentación.
Manejo y visualización de grafos	<code>NetworkX</code> vs. implementación propia pura	Implementación propia elegida para el núcleo (control total sobre la complejidad, requisito pedagógico del curso); <code>NetworkX</code> queda como candidata para visualización y para contrastar resultados en la Entrega 2.
Solver exacto escalable	<code>PuLP</code> vs. <code>OR-Tools</code>	<code>PuLP</code> preferido para la siguiente etapa: instala el solver CBC junto con el paquete (sin dependencias del sistema), API declarativa simple para la formulación ILP estándar de Vertex Cover, documentación suficiente para una formulación de este tamaño. <code>OR-Tools</code> se descarta por ahora por una curva de aprendizaje mayor para el beneficio adicional que aporta en instancias de este tamaño.
Datos y gráficos	<code>pandas</code> + <code>matplotlib</code>	Estándar de facto, documentación extensa, suficiente para el volumen de datos de este proyecto (decenas a cientos de filas).

Criterios generales aplicados a todas las decisiones: (1) facilidad de instalación sin dependencias del sistema operativo, (2) calidad y vigencia de la documentación oficial, (3) curva de aprendizaje razonable para el tiempo disponible, (4) adopción/soporte de la comunidad, (5) que no añada complejidad innecesaria al núcleo algorítmico (ver "Prioridad del proyecto", instrucción adicional 5 de la guía).

6 Métricas previstas para la comparación

- **Ratio de aproximación empírico:** $|C_{\text{algoritmo}}|/|C^*|$, calculado por `verify.approximation_ratio`. Es la métrica central: mide qué tan lejos queda cada algoritmo del óptimo en la práctica, más allá de su cota teórica.

- **Tiempo de ejecución** (ms), medido con `time.perf_counter()` tanto en `demo.py` como en `experiments/compare.py`.
- **Escalabilidad**: tiempo y ratio en función de $|V|$, para caracterizar cómo degrada cada algoritmo al crecer la instancia (ver resultados preliminares abajo).
- (Entrega 2) **Uso de memoria**, pendiente de instrumentar.

7 Resultados experimentales preliminares

Primera corrida de `experiments/compare.py`: grafos aleatorios $G(n, 0.3)$, $n \in \{6, 8, 10, 12, 14, 16, 18\}$, semilla fija por tamaño (reproducible).

n	$ E $	óptimo	greedy (ratio)	matching (ratio)	t_{exacto} (ms)
6	3	2	2 (1.00x)	4 (2.00x)	0.02
8	10	4	4 (1.00x)	6 (1.50x)	0.06
10	13	5	6 (1.20x)	10 (2.00x)	0.31
12	19	5	5 (1.00x)	10 (2.00x)	0.97
14	35	9	9 (1.00x)	12 (1.33x)	9.08
16	33	9	9 (1.00x)	14 (1.56x)	29.61
18	52	12	12 (1.00x)	16 (1.33x)	173.90

Observaciones preliminares:

- El tiempo del algoritmo exacto crece de forma visiblemente exponencial incluso en este rango pequeño (factor $\approx 6\times$ entre $n=16$ y $n=18$), confirmando la cota $O(2^V \cdot E)$ y justificando la necesidad de una alternativa ILP para la Entrega 2.
- El greedy iguala al óptimo en 6 de las 7 instancias; solo se desvía (ratio 1.2x) en $n=10$.
- El matching maximal se mantiene siempre dentro de su cota teórica (máximo observado: 2.0x, exactamente el límite superior), pero nunca superó al greedy en estas instancias.
- Estos resultados son preliminares (densidad fija $p = 0.3$, un solo tipo de grafo aleatorio); la Entrega 2 ampliará el experimento a distintas densidades y a la topología tipo backbone (Barabási-Albert) ya disponible en `data/instances/network_backbone_18.json`.

8 Conclusiones del avance

El diseño del sistema está implementado y verificado: estructura de grafo eficiente para redes dispersas, los tres algoritmos acordados funcionando y validados contra la traza manual del Avance 1, un plan de pruebas de 27 casos que cubre los escenarios básicos, límite y adversarios exigidos, y un primer experimento reproducible que ya deja ver el comportamiento esperado de cada algoritmo. El hallazgo del bug de no-determinismo (y su corrección) se documenta como evidencia concreta de aprendizaje autónomo y pensamiento crítico durante el desarrollo. Queda pendiente para la Entrega 2: incorporar el solver ILP como referencia escalable, ampliar la metodología experimental a más densidades y topologías, y optimizar la complejidad del greedy con una cola de prioridad.